

Tic-Tac-Toe

C++ Source Code • 2-Player Console Game

```
#include <iostream>
using namespace std;
/*
 * Tic-Tac-Toe Game (2 Players)
 * -----
 * - Board is 3x3.
 * - Player 1 uses 'X', Player 2 uses 'O'.
 * - Players take turns entering row and column.
 * - Game ends when someone wins or board is full.
 */
char board[3][3]; // Global board for simplicity
void initializeBoard();
void displayBoard();
bool placeMark(int row, int col, char mark);
bool checkWin(char mark);
bool isBoardFull();
int main() {
    char currentPlayer = 'X'; // Player 1 starts with 'X'
    int row, col;
    bool gameOver = false;
    initializeBoard();
    cout << "===== TIC-TAC-TOE =====\n";
    cout << "Player 1: X\nPlayer 2: O\n";
    while (!gameOver) {
        displayBoard();
        cout << "Player " << (currentPlayer == 'X' ? 1 : 2)
              << " (" << currentPlayer << ") enter row and column (1-3 1-3): ";
        cin >> row >> col;
        // Convert to 0-based index
        row--;
        col--;
        // Try to place the mark
        if (!placeMark(row, col, currentPlayer)) {
            cout << "Invalid move, try again.\n";
            continue; // Ask again without changing player
        }
        // Check if this move wins the game
        if (checkWin(currentPlayer)) {
            displayBoard();
            cout << "Player " << (currentPlayer == 'X' ? 1 : 2)
                  << " wins!\n";
            gameOver = true;
        }
        else if (isBoardFull()) { // If board is full and no winner
            displayBoard();
            cout << "It's a draw!\n";
            gameOver = true;
        }
    }
}
```

```

    }
    else {
        // Switch player: X -> O, O -> X
        currentPlayer = (currentPlayer == 'X') ? 'O' : 'X';
    }
}
return 0;
}
/*
 * initializeBoard
 * -----
 * Fills the board with spaces so it looks empty.
 */
void initializeBoard() {
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            board[i][j] = ' ';
}
/*
 * displayBoard
 * -----
 * Prints the current board state.
 */
void displayBoard() {
    cout << "\nCurrent board:\n";
    for (int i = 0; i < 3; ++i) {
        cout << " ";
        for (int j = 0; j < 3; ++j) {
            cout << board[i][j];
            if (j < 2) cout << " | ";
        }
        cout << "\n";
        if (i < 2) cout << "----+----+----\n";
    }
    cout << "\n";
}
/*
 * placeMark
 * -----
 * Puts mark ('X' or 'O') on board[row][col] if the cell is valid
 * and empty. Returns true if successful, false otherwise.
 */
bool placeMark(int row, int col, char mark) {
    // Check for valid range
    if (row < 0 || row >= 3 || col < 0 || col >= 3)
        return false;
    // Check if cell already occupied
    if (board[row][col] != ' ')
        return false;
    board[row][col] = mark;
    return true;
}

```

```

}
/*
 * checkWin
 * -----
 * Checks all rows, columns and diagonals for a win
 * for the given mark.
 */
bool checkWin(char mark) {
    // Check rows
    for (int i = 0; i < 3; ++i)
        if (board[i][0] == mark && board[i][1] == mark && board[i][2] == mark)
            return true;
    // Check columns
    for (int j = 0; j < 3; ++j)
        if (board[0][j] == mark && board[1][j] == mark && board[2][j] == mark)
            return true;
    // Check main diagonal
    if (board[0][0] == mark && board[1][1] == mark && board[2][2] == mark)
        return true;
    // Check other diagonal
    if (board[0][2] == mark && board[1][1] == mark && board[2][0] == mark)
        return true;
    return false;
}
/*
 * isBoardFull
 * -----
 * Returns true if there is no empty cell left.
 */
bool isBoardFull() {
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            if (board[i][j] == ' ')
                return false;
    return true;
}

```